

CSCI 6461 | TOPIC 2

RISC vs. CISC: Instruction Set Design Principles & Trade-offs

ISA choices, implementation costs, and performance consequences

Dr. J. Burns

Computer Systems Architecture

What Is an Instruction Set Architecture?

- An ISA is the programmer-visible specification of a processor.
- It defines instructions, registers, data types, addressing modes, and architectural state.
- Compilers generate code for the ISA; processors implement the ISA.
- ISA design therefore sits directly at the hardware/software boundary.
- A good ISA must remain useful for decades while implementations evolve underneath it.
- RISC and CISC are different historical answers to the question: **what should the ISA expose?**

Why ISA Design Matters

Software consequences

- Instruction count
- Code size
- Compiler complexity
- Calling conventions
- Binary compatibility

Hardware consequences

- Decode complexity
- Datapath organization
- Pipeline regularity
- Control logic
- Energy per instruction

ISA design redistributes complexity between software and hardware rather than eliminating it.

The Instruction-Set Design Problem

- Should one instruction perform a small primitive operation or a larger sequence of work?
- Should arithmetic instructions access memory directly or operate only on registers?
- Should instructions have one fixed length or many lengths?
- How many addressing modes should hardware understand?
- Should frequently used operations receive special instructions even if they complicate decode?
- RISC and CISC emerged from different answers shaped by the technology of their eras.

Historical Constraints Shaped Early ISAs

- Early main memory was expensive, slow, and physically limited.
- Compilers were less sophisticated than modern optimizing compilers.
- Hardware designers wanted programs to occupy fewer bytes and make fewer memory accesses.
- Rich instructions could express substantial work in a compact encoding.
- Microcode made it practical to implement complex instruction behavior using simpler internal steps.
- These pressures strongly encouraged what later became known as the CISC style.

The Origins of CISC

- CISC stands for **Complex Instruction Set Computer**.
- The philosophy favored rich instruction sets with many operations and addressing modes.
- Individual instructions could perform multi-step tasks such as memory access plus arithmetic.
- Variable-length encodings allowed common instructions to be compact while retaining expressive instructions.
- The goal was often to reduce program size and narrow the semantic gap between languages and machine code.
- The x86 ISA is the most prominent surviving example of a CISC lineage.

The CISC Design Philosophy

- Provide many instructions so software can request specialized operations directly.
- Permit operands to reside in registers or memory in many instruction forms.
- Support multiple addressing modes to make data structures convenient to access.
- Use variable instruction lengths to encode simple cases compactly and complex cases when needed.
- Allow one architectural instruction to correspond to several internal hardware actions.
- Accept more complicated decoding and control in exchange for expressive machine instructions.

CISC Example: More Work per Instruction

Conceptual example

Imagine an ISA instruction that computes $A = A + \text{memory}[B + 8]$.

- One architectural instruction identifies an address, reads memory, performs arithmetic, and writes a result.
- The instruction may encode an operation, base register, displacement, and destination.
- Fewer instructions are visible in the program.
- Internally, however, hardware may still perform address generation, memory access, addition, and write-back separately.
- The architectural instruction is compact semantically even if its implementation is multi-step.
- This distinction between **architectural instruction** and **internal work** matters throughout the lecture.

Why Complex Instructions Can Be Attractive

- Fewer instructions can reduce instruction-fetch traffic.
- Rich addressing modes can express common data-access patterns directly.
- Compact encodings improve code density, which can benefit instruction-cache behavior.
- Some specialized operations map naturally to common software idioms.
- Backward compatibility can preserve huge software ecosystems over many processor generations.
- The central attraction is not “complexity for its own sake” but doing useful work with fewer architectural instructions or bytes.

The Cost of Instruction Complexity

- Variable instruction length makes instruction boundaries harder to find quickly.
- Many addressing modes increase decoder and control complexity.
- Instructions may require very different execution times and hardware resources.
- Irregular operand rules complicate pipelines and dependency handling.
- Complex decode consumes area and energy before useful arithmetic even begins.
- As transistor budgets and compiler quality improved, architects questioned whether this complexity still paid for itself.

The Emergence of RISC

- RISC stands for **Reduced Instruction Set Computer**.
- Researchers observed that compiled programs used some instructions far more often than others.
- Many complex instructions were rarely generated by compilers or were not faster than simple sequences.
- Faster semiconductor technology made simpler hardwired control increasingly attractive.
- The idea was to optimize the common case rather than provide hardware for every conceivable operation.
- Berkeley RISC and Stanford MIPS helped establish this approach in the early 1980s.

The RISC Design Philosophy

- Keep instructions relatively simple and regular.
- Emphasize register-to-register arithmetic.
- Access memory explicitly through load and store instructions.
- Use relatively regular instruction encodings to simplify fetch and decode.
- Provide enough registers that temporary values can remain close to the execution units.
- Rely more heavily on compilers to combine simple instructions into efficient programs.

Berkeley RISC and Stanford MIPS

Berkeley RISC

- Regular instructions
- Large register emphasis
- Pipeline-friendly design
- Influenced SPARC

Stanford MIPS

- Simple load/store ISA
- Compiler-oriented design
- Pipeline simplicity
- Influential teaching model

The lasting contribution was a methodology: **measure real workloads, simplify the common path, and let compilers schedule simple primitives.**

RISC Principle 1: Simple, Regular Operations

- Regular operations reduce the number of special cases hardware must handle.
- Similar instruction formats allow common decode circuitry to be reused.
- Predictable operand placement simplifies datapath control.
- Shorter critical control paths can support higher clock rates or lower energy.
- Simplicity also makes pipelining easier because stages perform more uniform work.
- “Simple” does not mean weak: complex programs are built by composing many simple instructions.

RISC Principle 2: Load/Store Architecture

Key rule

Arithmetic operates on registers; separate instructions move data between registers and memory.

- A load computes an address and brings a value into a register.
- Arithmetic then consumes register operands.
- A store writes a register value back to memory when needed.
- This separates memory access from ALU computation in the ISA.
- The separation maps naturally onto pipeline stages such as EX and MEM.
- AArch64 follows this load/store model closely.

RISC Principle 3: Register-Based Computation

- Registers are much faster to access than main memory.
- Keeping active values in registers reduces expensive memory traffic.
- Register operands are easy to name with compact fields in an instruction.
- Multiple read and write ports can feed execution units efficiently.
- Compiler register allocation becomes important because software decides which values stay resident.
- This is a deliberate transfer of responsibility: simpler hardware requires smarter code generation.

RISC Principle 4: Regular Instruction Encoding

- Fixed or highly regular instruction lengths make boundaries easy to locate.
- Decoders can examine predictable bit positions for opcode and register fields.
- Instruction fetch can retrieve aligned groups without first discovering varying lengths.
- Regularity reduces front-end control complexity.
- The trade-off is that some operations may require more instruction bits or multiple instructions.
- AArch64 uses fixed 32-bit instruction encodings, a classic RISC-friendly property.

RISC Principle 5: Compiler-Friendly Design

- RISC assumes compilers can select and schedule simple operations effectively.
- The compiler exposes instruction-level opportunities rather than hiding work inside large instructions.
- Register allocation is essential because the ISA expects frequent register use.
- Regular instructions simplify cost modeling during optimization.
- Compiler improvements can therefore yield performance gains without changing hardware.
- The risk is that poor code generation can produce more instructions and more memory traffic.

Worked Example: Evaluate $z = (a + b) \times c$

Load/store-style sequence

```
ldr x1, [a]
ldr x2, [b]
add x3, x1, x2
ldr x4, [c]
mul x5, x3, x4
str x5, [z]
```

- The sequence is longer than a hypothetical memory-to-memory expression instruction.
- But each instruction performs a regular, easily identified job.
- This regularity is valuable for decoding, pipelining, dependency tracking, and compiler scheduling.

Hypothetical CISC Equivalent: Same Expression

One richer architectural instruction

`muladd_mem [z], [a], [b], [c]`

- This **hypothetical** instruction represents the same expression $z = (a + b) \times c$ in one architectural instruction.
- The instruction must obtain three source values from memory and ultimately write one result back to memory.
- Internally, the processor still needs address generation, multiple memory accesses, addition, multiplication, and a store.
- Architectural instruction count falls from six visible RISC-style instructions to one visible CISC-style instruction.
- The trade-off is a richer encoding and more complicated decode, sequencing, exception handling, and dependency behavior.
- The example illustrates CISC philosophy; it is **not** intended to claim that a real x86 instruction has exactly this form.

RISC vs. CISC: Same Computation, Different Exposure

RISC-style exposure

- Load operands
- Compute in registers
- Store result
- More visible instructions
- More regular operations

CISC-style exposure

- Memory may be an operand
- Richer single instructions
- Fewer visible instructions
- Variable work per instruction
- More decode/control complexity

Neither description alone predicts performance. We need the Iron Law.

Instruction Count: Is Fewer Always Better?

- A CISC program may require fewer architectural instructions for the same task.
- But instruction count says nothing about the time required by each instruction.
- A rich instruction may decode into several internal operations.
- A RISC program may use more instructions but execute them with high pipeline regularity.
- Code size depends on instruction width as well as instruction count.
- Therefore **fewer instructions is useful only when the other performance factors remain favorable.**

CPI: The Other Side of the Trade-off

- CPI is the average number of clock cycles per executed architectural instruction.
- A complex instruction may require more internal work and potentially more cycles.
- Regular instructions can make average execution behavior easier to pipeline.
- However, modern superscalar processors can complete multiple instructions per cycle, giving CPI below 1.
- CPI therefore reflects implementation, workload, cache behavior, and instruction mix.
- RISC versus CISC cannot be evaluated by instruction count without considering CPI.

Clock Cycle Time and Instruction Complexity

- The clock period must accommodate the critical timing requirements of the implementation.
- More complicated decode and control can lengthen front-end paths or require additional pipeline stages.
- Simpler instruction structure can reduce work required before execution begins.
- But modern processors often pipeline decode aggressively, so ISA complexity does not directly equal a slow clock.
- Extra pipeline stages can raise branch-misprediction penalties even when frequency increases.
- Again, one design variable cannot be considered in isolation.

The Iron Law Applied to ISA Design

$$T_{CPU} = IC \times CPI \times T_{cycle}$$

- **Instruction count (IC):** often favors richer instructions or denser encodings.
- **CPI:** often benefits from regular instructions and efficient execution pipelines.
- **Cycle time:** may benefit from simpler decode/control paths.
- ISA design can improve one term while harming another.
- Compiler quality and microarchitecture determine where the real balance lands.
- The Iron Law is the correct framework for performance arguments about RISC and CISC.

Worked Performance Comparison

Machine R

RISC-style program: 1.30 billion instructions, CPI 1.0, 3.0 GHz

Machine C

CISC-style program: 0.90 billion instructions, CPI 1.4, 2.8 GHz

$$T_R = \frac{1.30 \times 10^9 \times 1.0}{3.0 \times 10^9} = 0.433 \text{ s}$$

$$T_C = \frac{0.90 \times 10^9 \times 1.4}{2.8 \times 10^9} = 0.450 \text{ s}$$

Lower instruction count did not guarantee lower execution time.

Code Density and Memory Footprint

- Code density measures how much useful program behavior fits into a given number of instruction bytes.
- Variable-length CISC encodings can represent common operations very compactly.
- Fixed-width RISC instructions may consume more bytes for some instruction sequences.
- Larger code can increase instruction-cache pressure and memory bandwidth demand.
- ARM historically addressed code density partly through Thumb and Thumb-2 encodings; AArch64 uses fixed 32-bit instructions.
- Code density shows that regular hardware-friendly encoding has a real system-level trade-off.

Instruction Fetch and Decode Complexity

Regular fixed-width ISA

- Boundaries known immediately
- Predictable field locations
- Easier parallel decode
- Simpler alignment logic

Variable-length rich ISA

- Boundaries must be discovered
- Prefixes may alter meaning
- Fields vary by instruction
- More front-end logic required

Modern x86 processors invest substantial hardware in the front end precisely because compatibility with variable-length x86 instructions is valuable.

ISA Design and Pipelining

- Pipelines work best when instruction processing can be divided into repeatable stages.
- RISC regularity historically made five-stage pipelines comparatively straightforward to build.
- Separate load/store operations map naturally onto a dedicated memory-access stage.
- Fixed instruction length simplifies fetch and decode timing.
- Complex instructions can still be pipelined, but often require decomposition or specialized paths.
- Modern high-performance processors go far beyond classic pipelines, yet front-end regularity still matters.

ISA Design and Compiler Optimization

- Compilers transform high-level code into instruction sequences that exploit the ISA efficiently.
- RISC designs expose simple operations that compilers can reorder and combine.
- A large register set gives compilers more places to keep temporary values.
- CISC instructions may encode useful compound operations that reduce code size or instruction count.
- Modern compilers use detailed processor cost models rather than assuming one ISA philosophy is always superior.
- ISA quality therefore includes how well its features can be exploited by real compiler toolchains.

RISC vs. CISC: Energy and Thermal Trade-offs

RISC tendency

- Regular encodings can simplify fetch, alignment, and decode.
- Simpler architectural operations can reduce front-end work per instruction.
- Explicit load/store behavior can make data movement more visible and predictable.

CISC tendency

- Variable-length or richer instructions can require more complex decode machinery.
- One architectural instruction may perform several operations or memory accesses.
- Better code density can reduce instruction-fetch traffic and cache pressure.

Architectural implication

ISA style influences energy, but implementation and workload determine total efficiency.

Energy per Instruction Is Not Energy per Program

A useful first-order model

$$E_{\text{program}} \approx IC \times E_{\text{instruction}}$$

- A simpler instruction may consume less energy each time it is fetched, decoded, and executed.
- But a RISC sequence may require more architectural instructions to complete the same task.
- A denser CISC sequence may reduce instruction-cache accesses and front-end traffic.
- Memory behavior can dominate energy when instructions cause cache misses or DRAM accesses.
- The relevant metric is therefore total energy for the workload, not energy for one instruction.
- This mirrors the Iron Law: optimizing one factor in isolation can produce the wrong conclusion.

Power Becomes Heat — and Heat Requires Cooling

Energy and power

$$P = \frac{E}{t}$$

Higher switching activity, voltage, frequency, and active hardware structures increase power demand.

System consequence

architectural complexity



transistor switching



power consumption



heat generation



cooling requirement

- Cooling may range from passive heat spreaders to fans, liquid cooling, and data-center HVAC.
- Thermal limits can force frequency reduction, so power can ultimately constrain performance.

Performance per Watt: The More Useful Question

- Processor design increasingly optimizes useful work per unit of energy, not performance alone.
- Simpler decode and regular execution can help efficiency, especially in constrained systems.
- Code density, cache behavior, speculation, vector units, and memory traffic can reverse simple ISA-level expectations.
- Process technology, voltage, frequency, and microarchitecture often matter more than the RISC/CISC label.
- ARM's success now spans phones, laptops, and cloud servers, reflecting sustained attention to efficient implementations and system design.
- Therefore, “ARM is efficient because RISC is simpler” is an incomplete architectural explanation.

Architectural implication

Compare **energy per workload**, **performance per watt**, and **thermal constraints**; ISA labels are only part of the evidence.

Why Modern x86 Looks RISC-Like Inside

- Modern x86 cores preserve the complex x86 ISA for software compatibility.
- The front end decodes many x86 instructions into simpler internal micro-operations (μ ops).
- Those μ ops are scheduled onto regular execution resources inside the core.
- This separates a complex architectural interface from a more regular internal execution model.
- Some x86 instructions decode into one μ op; others require several or specialized handling.
- The lesson: **ISA style and microarchitecture style are not the same thing.**

Why Modern RISC ISAs Are Not Necessarily “Simple”

- Modern RISC ISAs include SIMD, cryptography, virtualization, atomics, and system-control instructions.
- AArch64 has many instruction classes and sophisticated addressing features.
- High-performance ARM cores use branch prediction, out-of-order execution, speculation, and large caches.
- None of that violates the central RISC ideas of regular encoding and load/store organization.
- “Reduced” should be understood historically as architectural regularity, not a tiny instruction count.
- Calling a modern ARM core “simple” would confuse ISA philosophy with implementation sophistication.

RISC Beyond Phones: Laptops

- Modern RISC is no longer confined to embedded controllers or mobile phones.
- Apple Silicon brought Arm-based processors into mainstream laptop and desktop computing.
- Current MacBook Pro systems use Apple M5-family chips for high-performance portable computing.
- Apple emphasizes both performance and battery life; its MacBook Pro overview lists up to 24 hours of battery life.
- This makes laptops a useful example of the performance-per-watt argument, not just the low-power argument.
- The architectural lesson is that RISC principles can scale from constrained devices to high-performance personal computers.

Source: Apple MacBook Pro overview and technical specifications.

RISC in the Data Center: Google Axion

- Cloud providers now deploy Arm-based processors for general-purpose server workloads.
- Google Axion is a custom Arm-based CPU designed for Google Cloud data-center workloads.
- Google identifies web and app servers, databases, analytics, media processing, and CPU-based AI as Axion target workloads.
- Google reports production deployment for services such as Bigtable, Spanner, BigQuery, Blobstore, Pub/Sub, Google Earth Engine, and YouTube Ads.
- These are not small embedded workloads; they are latency-sensitive and throughput-oriented cloud services.
- The architectural question becomes performance per watt, fleet-level cost, cooling, and software ecosystem support.

Source: Google Cloud Axion announcement and C4A deployment notes.

Data-Center Workloads: What Runs on Arm?

Storage and databases

- Spanner
- Bigtable
- Cloud SQL / AlloyDB-class workloads

Serving and platforms

- YouTube Ads platform
- web and application servers
- containerized microservices

Analytics and pipelines

- BigQuery
- Dataflow-style pipelines
- batch and containerized jobs

AI and inference

- CPU-based inference
- preprocessing and data movement
- services where accelerator use is not dominant

ARM AArch64 as a Modern RISC ISA

- AArch64 uses fixed 32-bit instruction encodings.
- Most arithmetic and logical operations use register operands.
- Memory is accessed primarily through explicit load and store instructions.
- Thirty-one general-purpose integer registers support register-oriented computation.
- Instruction formats are structured to make common fields relatively regular.
- These properties make AArch64 an excellent architecture for studying the connection between ISA and datapath design.

Preview

Next we move from philosophy to the actual AArch64 programmer's model and assembly syntax.

RISC vs. CISC: What Actually Matters Today?

- The binary labels are less predictive than they were in the 1980s.
- Modern CISC processors use sophisticated internal decomposition and execution engines.
- Modern RISC processors contain highly complex microarchitectures and rich ISA extensions.
- Compatibility, code density, compiler ecosystem, implementation quality, and workload matter enormously.
- The enduring RISC contribution is emphasis on regularity, load/store organization, and compiler-visible simple operations.
- The enduring lesson is to evaluate architectural choices quantitatively rather than by slogans.

Synthesis: ISA Choices Are Trade-offs

- CISC tends to expose richer operations and often achieves strong code density.
- RISC tends to expose more regular operations that are easier to decode and pipeline.
- Neither lower instruction count nor higher clock rate guarantees better performance.
- The Iron Law forces us to consider instruction count, CPI, and cycle time together.
- Modern processors blur the historical boundary while preserving very different architectural interfaces.
- AArch64 gives us a modern RISC case study for the rest of the course.

Design question

Do not ask “Which philosophy is faster?” Ask: **what trade-off does this ISA choice create, and what does the measured workload say?**